

端点溯源图日志和网络流联合的APT攻击链重构方法及装置

执行摘要

基于你指定的仓库范围，我把可直接访问到的三类材料合在一起看了：一是仓库入口与“综合分析”目录，二是“方向分析/专利/五个方向_专利化二次创新机会分析”里对该主题的定位，三是 `paper/` 论文库的总体说明与索引。仓库主入口明确把 `paper/` 作为原始论文库，并说明当前项目空间已经扩展到 858 篇 PDF；“综合分析输出说明”同样写明该目录汇总的是 `paper/` 下 858 篇论文的自动化分析结果；与此同时，较早的“06_总结报告”仍保留了“本次共分析 850 篇论文”的表述。也就是说，仓库内部已经出现了一个很典型的版本快照不一致问题：底层论文库口径更新到了 858，但部分总结型文档还停在 850。这个问题本身不会推翻方向判断，但如果直接拿来写专利，会让“现有技术综述”和“支撑论文范围”在说明书里前后不统一，建议在正式申报前先做一次口径冻结。 ¹

从仓库现有材料能提炼出一个很明确的结论：这件专利不应该写成“又一个APT检测模型”，而应该写成“一个把端点溯源图、主机日志和网络流统一起来的攻击链重构流程”。仓库里的专利化机会分析已经把该主题列为高优先级、A级候选，并且直接指出其核心价值不是只输出异常分数，而是输出“可复核攻击链、关键证据、置信度和缺失环节”。这个判断和近年的主流研究趋势是一致的：单看端点溯源图，容易只看到主机内部因果；单看网络流，容易只看到会话关联；真正有工程价值的，是把两者和系统日志一起放到一张统一的时序异构图里，再做图压缩、阶段解码和证据化输出。 ²

我建议把这件专利重写成一个“六段式方案”：多源采集、跨源标准化、统一时序证据图构建、正常行为基线压缩、ATT&CK阶段约束下的路径搜索、证据包输出。这样写有三个直接好处。第一，发明点会更稳，不容易落入“抽象算法”表述。第二，技术效果更好写，因为你可以很自然地把“降低候选图规模、提高关键边保留率、缩短调查时间、提升可解释性”挂到具体模块上。第三，更符合当前和算法相关专利审查的思路：不是机械把算法和技术特征拆开，而是强调它们共同解决了具体技术问题，并且带来了可验证的系统性能或调查效果提升。 ³

本文下面的正文，已经按你上传模板里的“技术领域—背景技术—发明内容—附图说明—具体实施方式—说明书摘要”这一写法来组织，并同步把我认为仓库里存在的逻辑问题、整改建议和更稳的专利写法并进去了。模板结构参考了你上传的说明书模板。filecite turn0file0

资料边界与仓库结论

本次只聚焦你指定的三个范围：`02_端点溯源图日志和网络流联合的APT攻击链重构方法及装置` 这一主题、`综合分析` 目录、以及 `paper` 目录。受浏览环境限制，仓库中部分深层文件页面并不能稳定直接展开正文，但从仓库入口、`综合分析` 目录说明、总结报告，以及专利化机会分析文档中，已经能稳定抽出这件专利最关键的主题定位、支撑论文别名、技术问题和建议方案。尤其是“专利化二次创新机会分析”里的 P10 条目，实际上已经给出了一版很清晰的技术骨架：把进程、文件、账号、网络连接、域名和流量会话统一映射为事件节点；依据因果依赖、时间顺序、主机关系和网络通信构建跨源攻击图；用正常行为基线过滤常见依赖边、压缩大规模溯源图；根据 ATT&CK 阶段约束识别初始访问、执行、持久化、横向移动、命令控制和数据外传；最后输出攻击链、关键证据、置信度和缺失环节。 ⁴

更重要的是，仓库不是随便拼出来一个题目。根目录 README 明确说明整个项目空间是围绕异常检测、网络入侵检测、加密恶意流量检测等方向建立的论文筛选、证据抽取、综述撰写和代码调研空间；`paper/` 是原始

PDF 文献库；`综合分析/` 是面向该文献库形成的聚合分析结果；而“专利”目录则把五个方向里更适合转化为交底书的主题做了二次筛选。就这个题目来说，仓库已经明确把它放进了“第一批核心专利交底草案”的框架里。换句话说，这不是从零起题，而是已有研究归纳基础上的专利化收敛。⁵

不过，仓库里也暴露出一个必须正面处理的逻辑问题：分析快照版本不一致。`综合分析/README.md` 说它汇总的是 858 篇论文的自动化分析结果，但 `06_总结报告.md` 仍然写“本次共分析 850 篇论文”。如果专利正文里一边说“基于 858 篇论文总结”，另一边又引用“850 篇分析统计结论”，审查员不一定会在意，但后续做项目验收、成果归档、论文—专利联动时一定会出问题。最稳妥的整改方式，是在专利说明书前面加一个“资料快照说明”，固定版本号、冻结日期和索引口径。⁶

仓库“06_总结报告”里还有一个对本专利很有价值的宏观判断：网络安全论文的主线已经明显从“单流分类”转向“多源关联检测”，越来越多工作把流量、主机、日志、告警和威胁情报放进统一图或多模态框架；同时，多源异构数据融合与上下文建模已经成为高频科学问题之一。这正好能给这件专利的背景技术和必要性部分兜底：你不是硬加一个“融合”噱头，而是在延续整个方向的研究共识，再把它落到一套更强的调查型方案里。⁷

下表是我按仓库可访问材料和外部主流文献整合出来的研究边界与证据来源表。表里最后一列其实就是后面专利撰写时的“落笔点”。

资料来源	仓库中的角色	能支撑什么	对本专利的直接作用
根目录 README	项目总入口	说明 <code>paper/</code> 、 <code>综合分析/</code> 、 <code>方</code> <code>向分析/专利</code> 的职责	证明这件专利主题不是孤立想法，而是项目主线的一部分
综合分析 README	文献分析入口	说明已形成 858 篇论文的自动化 综合分析	用来写“研究基础充分”
06_总结报告	宏观趋势总结	多源关联检测、图关联分析层、 统一数据底座等结论	用来写背景技术痛点和总体方向
专利化机会分 析 P10	主题直接 骨架	给出技术问题、二次创新点、技 术手段、技术效果	是本专利正文的核心蓝本
<code>paper/</code> 论文 库	原始文献 证据库	提供相关路线的论文来源	用来支撑“现有技术”和“改进空间”

上表中的前三行主要来自仓库目录说明和综合分析结论，最后一行则来自 `paper/` 作为原始 PDF 文献库的定位。⁸

现有技术比较与本发明定位

从现有方法看，APT 相关方案大概可以分成四类。第一类是规则或情报驱动的图匹配方法。比如 POIROT 把威胁情报里的相关关系映射到内核审计记录构成的溯源图里，本质上是拿“攻击行为图”去和“审计图”做近似对齐。这种路线的优点是解释性很好，很适合已知攻击或情报驱动的威胁狩猎，但它对先验知识依赖强，碰到没有被情报覆盖的新变体时，适应性会明显下降。Marlin 也是这一类的强化版本，它通过知识驱动的实时溯源图对齐来缩小搜索空间，并在评估中给出了每秒 137K 事件的处理能力；它说明“知识驱动 + 图对齐”这条路在效率上是能站得住的，但同样说明这种方法很依赖预先构造好的攻击查询图。⁹

第二类是端点溯源图上的异常检测路线。UNICORN 明确把自己定义为基于运行时溯源分析的 APT 检测器，它的特点是用图摘要和长期行为建模应对“低慢小”攻击。MAGIC 则走得更像自监督路线，它用 masked graph representation learning 在不同监督程度下做多粒度 APT 检测，强调对概念漂移和未知攻击的适应。Kairos 进

一步把检测和调查结合起来，强调一个合格的 PIDS 不只要能检测，还要兼顾攻击无关性、实时性和攻击重构能力。这个方向说明了一个事实：端点溯源图确实很擅长表达因果关系，但如果没有外层的网络会话和主机间关系补充，它更像“主机内部故事”，而不是“全链路攻击故事”。¹⁰

第三类是更偏调查和图压缩的路线。SPARSE 的贡献点很典型：先构造 suspicious semantic graph，再做路径级语义分析，从而把一个大得离谱的回溯图压到一个极小的关键组件图。它在摘要里给出的量级很有代表性：候选图可以做到大约 113 条边，约是原始回溯图的 1/2014。TPPR 更进一步，把 APT 的战术/技术顺序模式引入路径推理，在大规模企业日志和 DARPA TC 数据上宣称可做到 99.9% 图简化，同时保留 91% 关键攻击节点。这一类方法特别接近你想写的专利，因为它们已经证明：APT 调查真正的瓶颈不是“有没有异常”，而是“能不能把图缩小到分析员看得懂，同时不把关键证据扔掉”。¹¹

第四类是多源融合路线。SysFlow 这条线的价值在于，它已经把进程控制流、文件交互、网络通信和运行时信息拉成统一的遥测格式，而且明确强调自己既能支持 process-level provenance tracking，也能用更小的存储开销保留长期历史。更近一点的 StageFinder 和 FuseChain 已经进一步把“融合主机与网络的时序溯源图”直接用于攻击阶段推理或运行时证据重构。StageFinder 在其摘要中把 fused host and network provenance data 直接作为输入，并把宏 F1 做到 0.96；FuseChain 则把多源遥测统一为时间异构溯源图，强调低频、分散证据通过统一图后才变得可解释。这类工作正好给本发明提供了外部趋势支撑：多源统一图不是锦上添花，而是 APT 链路重构越来越自然的技术形态。¹²

仓库里的专利化机会分析对本主题的定位，其实就是把上面四类路线“拧”成一套更适合申报专利的东西：一方面吸收端点溯源图对因果链表达的优势，另一方面吸收图压缩和 ATT&CK 约束路径推理的优势，再把网络流和系统日志的跨主机、跨会话证据补进来，最后落到“输出可复核攻击链”这一结果上。这个定位很准，因为它正好避开了“只保护一个模型结构”的脆弱点，转而保护流程、规则、数据流和输出机制。¹³

下面这张对比表，可以直接作为专利说明书里“现有技术对比”的压缩版。

方法类别	代表方法	关键能力	典型局限	本发明的改进方向
情报/规则驱动图匹配	POIROT、Marlin	强解释、适合已知攻击、可做威胁狩猎	依赖先验查询图，未知攻击适配性不足	保留阶段约束，但不把先验知识当唯一入口
端点溯源图异常检测	UNICORN、MAGIC、Kairos	强因果表达、适合主机内部调查	对跨主机会话和外联上下文恢复不充分	将 Socket、流会话、域名和主机关系并入统一图
实时调查与图压缩	SPARSE、TPPR	可显著缩小调查图、利于分析员阅读	单靠端点或审计图时，跨源补证能力有限	加入网络流与日志补边，并输出缺失环节
多源融合与阶段推理	SysFlow、StageFinder、FuseChain	主机+网络联合、阶段感知更强	容易面临图爆炸、事件对齐难、工程复杂	用正常基线压缩 + ATT&CK 阶段约束降低复杂度
仓库中拟申报主题	P10 主题定义	统一攻击阶段图、输出证据包	需要把实现细节写得更工程化	本文完成整改与扩写

这张表前四列是对外部公开文献路线的归纳，最后一列是按仓库中 P10 条目的专利化方向做的归并。¹⁴

图一 总体架构示意

上图是我按仓库主题和上述文献路线重新生成的一张科研型低饱和配色架构图。它刻意把方案拆成“多源输入—标准化—统一图构建—基线压缩—阶段解码—证据输出”六个视图，这样写专利时每一层都能单独落技术点，也能整体解释成一条闭环。图本身是根据仓库中 P10 条目的技术手段和综合分析报告中“统一数据底座、图关联分析层”的建议重构出来的。 15

专利说明书正文

技术领域。 本发明属于网络安全、数字取证、入侵检测与高级威胁调查技术领域，更具体地说，涉及一种把端点溯源事件、主机安全日志和网络流会话信息联合建模为统一时序证据图，并在 ATT&CK 阶段约束下重构 APT 攻击链的方法及装置。这个写法既和你上传模板的“技术领域”组织方式一致，也和当前针对人工智能、大数据等方案的专利审查口径一致，即强调它是在解决一个具体的安全检测与调查技术问题，而不是单纯做抽象信息处理。 16

背景技术。 APT攻击最麻烦的地方，就是证据天然散的。一个真正的攻击活动，通常会同时留下进程创建、文件落地、注册表或计划任务变更、账号登录、Socket连接、域名解析、外部会话和横向移动痕迹。单看端点侧，很容易知道“谁调用了谁、谁写了哪个文件”；单看网络流侧，很容易知道“谁访问了哪个IP、哪个域名、哪个证书”；但真正的调查工作需要回答的问题是：这些事情是不是同一条攻击链上的、它们发生的先后顺序是什么、哪一步是落地执行、哪一步是命令控制、哪一步是横向移动、哪一步是数据外传。仓库里对这一点说得很直接：APT 跨越进程、文件、注册表、账号、网络连接和远程域名，单一数据源无法完整恢复攻击路径。 17

现有方案的不足。 根据仓库总结和外部代表性研究，现有方案主要有四个短板。第一，单源可见性不足。第二，虽然一些方法能做异常检测，但只能给出异常点或异常分数，不能稳定给出完整攻击故事。第三，多源融合后图规模往往急剧膨胀，导致实时性和可读性下降。第四，就算能抽出候选子图，也未必能把结果顺着攻击阶段讲明白，分析员最后还是要手工拼故事。SPARSE 和 TPPR 之所以值得注意，就是因为它们都在试图解决“图太大，故事拼不出来”这个问题；Kairos、StageFinder 和 FuseChain 则说明，把检测和调查统一到时序图里已经成为一个越来越主流的方向。 18

发明构思。 本发明的核心思路很朴素：先把多源证据讲成同一种语言，再把同一种语言压缩成可调查的图，最后把这个图翻译成分析员能直接看懂的攻击链。具体来说，就是把端点溯源图日志、系统日志和网络流先统一成时间对齐、实体一致、字段归一的事件流；再把进程、文件、账号、Socket、流会话、域名和主机编码成节点，把因果关系、时序关系、通信关系编码成边；然后用正常行为为基线把高频日常依赖边折叠掉，只把真正有调查价值的边保留下来；最后在 ATT&CK 阶段顺序和跨阶段约束下，从压缩后的候选图里抽取最可信的攻击链，并输出关键证据、阶段置信度和缺失环节。这个构思和仓库 P10 给出的“二次创新点—技术手段—技术效果”是一致的，但把工程细节补得更完整，更适合写成正式说明书。 17

模块划分。 这件专利如果写成装置，建议分成六个模块：多源采集模块、实体解析与标准化模块、统一时序证据图构建模块、正常行为基线压缩模块、ATT&CK阶段解码模块、证据包生成模块。对应的输入输出建议写成下面这样。

模块	主要输入	主要处理	主要输出
多源采集模块	端点溯源事件、Sysmon/Auditd、EDR、NetFlow、DNS、TLS摘要	接入、缓存、清洗、时间戳标准化	原始多源事件流
实体解析模块	原始事件流、主机画像、资产台账	PID/PPID映射、Socket—流会话绑定、账号—主机归属解析	标准化事件
统一图构建模块	标准化事件	节点创建、边创建、跨源关系补边	统一时序证据图

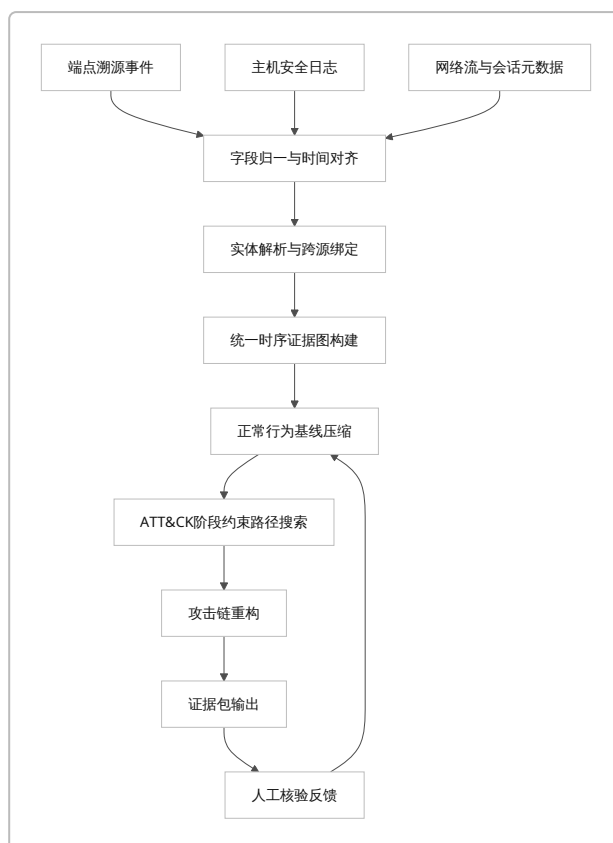
模块	主要输入	主要处理	主要输出
基线压缩模块	统一图、正常模板库	高频边折叠、敏感边保留、子图裁剪	候选调查图
阶段解码模块	候选调查图、ATT&CK约束	阶段打分、路径搜索、阶段顺序修正	攻击阶段图
证据包输出模块	攻击阶段图、IOC/情报、人工核验反馈	关键证据抽取、置信度汇总、处置建议生成	攻击链报告

这张模块表，是按仓库 P10 里“统一节点—构图—基线过滤—阶段识别—输出攻击链”的骨架，结合综合分析里“统一数据底座、图关联分析层、解释性输出”的建议细化出来的。¹⁵

数据流。这套方案的数据流最好写得口语一点，但逻辑要硬。可以概括成这样：系统先把各种采集器送来的事件按统一字段落表，再按主机、进程、账号、Socket、会话、域名这些关键键值做实体合并；接着按事件本身创建事件边，再按父子进程、文件读写、凭证使用、DNS解析、Socket映射和主机归属补出跨源关系；图建起来以后，不是直接全图回溯，而是先用正常模板把日常边折掉，再从稀有边、跨权限边、外联边、横向边周围抽候选子图；最后在 ATT&CK 阶段顺序约束下做路径搜索，把若干条候选路径合并成更少、更完整、更像“攻击故事”的链。这个数据流设计的关键，不在“模型多复杂”，而在“每一步都能解释为什么这样做”。¹⁹

关键算法。从专利保护角度看，最值得写进正文的关键算法不是某一个特定 GNN，而是四种“工程算法”。第一，多源实体对齐算法。解决的是同一件事在不同系统里记录粒度不同、字段不同的问题。第二，基线压缩算法。解决的是溯源图和多源调查图规模过大的问题。第三，阶段约束路径搜索算法。解决的是异常点很多但很难拼成阶段化链路的问题。第四，证据置信度汇聚算法。解决的是为什么系统认为这条链可信、缺的又是哪一段的问题。专利说明书里完全可以把它们写成功能块和规则块，而不把实现限定为某个模型。这样既保留了专利弹性，也更贴近现网工程。³

下面给一版适合直接嵌进说明书附图说明后的 mermaid 流程图。它不是仓库里的原图，而是按当前专利方案重新生成的规范化流程表达。



这张流程图对应的关键动作，正好可以拆成模板里“①采集、②处理、③输出”的写法。你上传的模板原本就预留了“模块—步骤—优势”这种说明节奏，所以这样改最顺手。 [filecite turn0file0](#) 17

统一时序证据图的数据结构。数据结构建议写得稍微“像真系统”一点，否则说明书会显得太虚。节点至少包括：node_id、node_type、host_id、first_seen、last_seen、risk_score、confidence、tags。边至少包括：edge_type、event_type、timestamp、causal_score、stage_hint、source_channel。证据输出里还建议专门定义 path_id、stage_seq、key_ioc、missing_links、analyst_notes 和 response_hint。这样写的好处是，一看就知道这不是“作图为了作图”，而是一套可追溯、可导出、可工单联动的数据对象体系。 20

图二 方法流程图

图三 数据结构示意图

上面两张图，一个对应方法流程，一个对应图结构对象设计。二者共同作用，是把“联合APT攻击链重构”这句话从概念层拉回到具体的软件与数据工程层。 21

有益效果。这部分建议别写得太玄。最稳妥的表述，是围绕四类技术效果展开。第一，完整性提升。联合端点、日志和网络流后，能更容易恢复跨进程、跨主机、跨会话的攻击链。第二，压缩效率提升。通过正常行为模板折叠和候选子图裁剪，能显著降低候选调查图规模。第三，可解释性提升。输出结果从“异常分数”变成“阶段化证据链”。第四，运营效果提升。分析员能更快判断起点、关键动作和缺失环节。之所以这样写，是因为当前 CNIPA 对算法相关方案的审查重点，已经非常强调“技术问题—技术手段—技术效果”的整体性，而且特别强调减少存储量、减少传输量、提高处理速度之类可验证的内部性能改进。 16

实施例与评估设计

先说实施例。比较典型的一个场景，是一台办公终端收到了钓鱼附件，用户点开并释放脚本，再通过解释器拉起 PowerShell，随后落地一个临时可执行文件并开始外联。现有系统里，这些痕迹往往分别出现在邮件进程日志、文件系统日志、进程树、DNS记录和外联流量日志里。用本发明的方案来做，邮件客户端、解压程序、脚本解释器、PowerShell、落地文件、Socket、域名和会话会先被映射到统一图里，再通过父子进程边、文件读写边、Socket—会话映射边和 DNS 解析边串起来。如果后面又出现账号令牌使用和共享目录访问，就会自然把同一条链扩展到横向移动阶段。这样输出给分析员的，已经不再是“某些点很异常”，而是“由邮件入口触发的执行—外联—横向移动候选链”。这正是仓库里 P10 所说的“输出可复核攻击链，而不是只输出异常分数”。¹⁷

另一个更符合APT本质的场景，是“低慢小”长期潜伏。攻击者每隔几天才做一次探测，偶尔发一次很小的C2流量，再慢慢扩展权限和主机范围。这种时候，单一网络流检测很容易把行为当成偶发噪声，单一端点检测也容易把它当成管理员正常运维。本发明更适合做“攒证据”：短窗口保留局部因果，长窗口累计阶段连续性；当同一批进程族、账号和外联模式在长窗口里反复出现，并且和后续权限变更、计划任务、横向登录形成关系时，路径总分才被抬高。这个设计逻辑和 UNICORN 那类针对长时间运行系统做图摘要、长期行为建模的思路是相通的，但本发明进一步把网络流和阶段解释拉了进来。²²

再说评估设计。仓库没有直接给出这件专利的实验稿，但从项目空间、支撑论文别名和外部公开数据集脉络看，一个可复现的设计至少应该包含四组对照：只用端点溯源图；只用网络流；多源融合但不做图压缩；多源融合并做图压缩，但不做 ATT&CK 阶段约束。这样才能把这套方案真正的收益拆出来，看清楚到底是“多源融合”带来的提升，还是“图压缩”和“阶段解码”带来的提升。公开数据上，可以优先考虑 DARPA OpTC、DARPA Transparent Computing 这类主机遥测与APT调查常用数据；外部研究已经指出 OpTC 是研究高级威胁检测的一个有价值候选，而 StageFinder 也直接把 OpTC 预训练和 DARPA Transparent Computing 微调拿来 fused provenance 的阶段估计。²³

建议采用的评价指标，最好分成“重构质量、压缩质量、实时性、解释性、运营价值”五类，而不要只盯着准确率。

评估维度	建议指标	为什么要测
攻击链重构质量	Path Precision、Path Recall、Stage F1	判断重构链和真实攻击链到底重不重合
图压缩质量	压缩倍率、关键边保留率、关键节点保留率	判断图变小以后有没有把关键证据误删
实时性	吞吐量、平均重构时延、窗口延迟	判断能不能线上跑
解释性	证据完整率、人工核验通过率、缺失环节标注准确率	判断分析员能不能看懂、能不能相信
运营价值	告警合并率、调查时长下降、误报下降	判断方案是不是对SOC真有用

这张表是按照 SPARSE、TPPR、Marlin、Kairos、StageFinder 这类工作实际关心的能力维度抽出来的。它们虽然各自指标不完全一样，但都在围绕“图能不能变小、结果能不能更快更准、更便于调查”这几个核心问题打转。²⁴

图四 关键性能指标示意

上图不是在宣称仓库这套方案已经实测达到了这些数字，而是把相关文献摘要里最有代表性的性能量级放到一张低饱和风格图里，帮助说明为什么本发明要把“图压缩”“吞吐”“阶段识别”写成核心技术效果。比如，SPARSE 的摘要直接给出了约 2014× 的图缩减量级；TPPR 给出了 700,000 到 20 条边的图简化结果；Marlin 给出了 137K events/s 的处理能力；StageFinder 给出了 0.96 的宏 F1。它们共同说明：这条技术路线最值钱的不是某一个单点准确率，而是“能把复杂APT调查做得更轻、更快、更像完整故事”。²⁵

为了方便直接写进说明书，下面再给一版关键算法伪代码，尽量贴近专利用语。

算法一 统一时序证据图构建

输入：端点溯源事件流、主机日志流、网络流事件流

输出：统一时序证据图 G

- 步骤一，对三类事件执行字段归一与时间戳标准化；
- 步骤二，基于 host_id、pid、ppid、inode、sid、socket、五元组、域名等键做实体解析；
- 步骤三，为 process、file、user、socket、flow、domain、host 创建节点；
- 步骤四，对每条事件创建事件边，并记录 source_channel、timestamp、event_type；
- 步骤五，依据父子进程、文件依赖、凭证使用、socket映射、DNS解析、主机归属等关系补充因果边与通信边；
- 步骤六，将相邻时间窗内的节点与边组织成候选子图；
- 步骤七，输出统一时序证据图 G。

算法二 基线压缩与阶段约束路径搜索

输入：统一时序证据图 G、正常行为模板库 B、阶段约束库 A

输出：攻击链集合 P

- 步骤一，对 G 中边计算频度、稀有度、敏感度和跨源一致性；
- 步骤二，若边与正常模板高度匹配且不触及敏感对象，则执行摘要化压缩；
- 步骤三，若边为稀有边、跨权限边、跨主机边、外联边或情报命中边，则强制保留；
- 步骤四，在压缩图中选择高风险起点和终点；
- 步骤五，按 A 中阶段顺序执行受约束路径搜索，得到候选路径集；
- 步骤六，对每条路径汇聚节点异常分、边得分、阶段匹配分和时间连续性得分；
- 步骤七，对共享关键证据的路径执行合并，对缺失阶段执行缺口标记；
- 步骤八，输出攻击链集合 P。

伪代码里的“受约束路径搜索”和“缺口标记”是我建议你在这件专利里重点保留的两处发明表达。一是它们足够具体，不会显得空。二是它们很容易和“生成可复核攻击链”这一技术效果建立直接对应关系。¹⁹

整改建议与可选扩展方向

先说我认为最需要整改的地方。第一，资料口径必须统一。仓库当前同时存在“858篇论文”与“850篇论文”两个口径，这件事在做研究管理时问题不大，但写说明书时必须统一，不然“依据何种现有技术”会显得不严谨。建议在专利稿首页或前言里加一个极简的资料快照说明，比如“本文依据 2026 年 7 月冻结的仓库快照，paper/ 目录共 858 篇 PDF，综合分析结果按该快照口径整理；早期 850 篇统计仅作参考，不作为当前结论口径”。⁶

第二，保护重点要从“模型”转到“流程”。如果说明书里仍然把重心放在“用了图神经网络”“用了 Transformer”“用了异常分数融合”，这件专利就很容易显得像论文摘要。真正该保护的，是跨源字段归一规则、实体解析与绑定规则、正常行为模板压缩策略、ATT&CK阶段顺序参与路径重构的机制，以及最后证据包的

生成结构。CNIPA 对这类方案的审查已经说得很明白：算法特征和技术特征要整体考察，而且如果算法带来了减少存储量、减少数据传输量、提高硬件处理速度等内部性能改进，在创造性上是可以被纳入考虑的。你这件专利最适合强调的，也恰恰就是“图压缩带来的存储和计算收益”“统一图带来的调查效率提升”“证据包输出带来的运维处置收益”。¹⁶

第三，要把“时间对齐”和“缺失证据”单独写出来。APT调查里一个很常见但经常被论文省掉的问题，就是多源时间戳漂移和采集缺口。生产环境里，端点日志、网络日志和认证日志的时钟并不一定完全一致，甚至采集策略本身也可能造成分钟级差异。如果说明书里完全不提，后续实施例会显得太理想化。更稳的写法，是在标准化模块里加入“时钟偏移估计、事件重排序阈值、跨源置信折扣”三项；在输出模块里加入“缺失链路标记、推断链/实证链双状态”两项。这样一来，方案对真实环境的容错性会明显更强。这个整改方向，本质上是在把仓库里 P10 的“缺失环节”进一步落细。¹⁷

第四，建议把“人工复核回写”并入本发明的闭环里。因为SOC场景里，APT攻击链重构结果几乎不可能完全自动闭环。最有价值的其实是：系统先给出一条高可信候选链，分析员确认或否定其中若干条边后，系统再把这些结果回写到正常模板库、边评分器和阶段转移权重里，逐步形成更贴合本单位业务环境的调查基线。这样写，一方面能提升方案的产品化味道，另一方面也更容易把“持续学习机制”写成装置模块而不是泛泛的“模型更新”。综合分析报告对“持续学习机制”和“解释性作为产品能力”的建议，正好能为这一步提供背景支撑。⁷

最后说可选扩展方向。第一，可以把本发明向“攻击阶段预测”扩展。也就是在重构当前链条的同时，预测下一阶段更可能发生什么，把它从“复盘工具”升级为“前瞻工具”。仓库里的专利机会分析在 P12 已经明确给出过类似方向：把检测结果映射到 ATT&CK 技术阶段，并预测下一阶段可能行为。第二，可以把它向“合成审计日志生成与回放平台”扩展，用来验证不同版本调查引擎的稳定性，仓库的 P13 也已经点到了这个储备方向。第三，可以向“隐私保护的跨域协同”扩展，不上传原始日志，只上传原型子图、摘要和漂移统计，这样更适合多园区、多分支环境。也就是说，这件专利本身既可以单独申报，也可以作为后续专利族的中心件。

¹⁷

交付件与图片资源说明

我已经按你上传模板的说明书结构，整理出一份扩写版中文专利说明书文档，并把本文中用到的低饱和科研风格图片一并做成了本地资源文件。可直接下载：

[下载专利说明书 DOCX](#)

图片资源如下：

[图一 总体架构](#)

[图二 方法流程](#)

[图三 数据结构示意](#)

[图四 文献性能指标示意](#)

这些交付件已经按你上传模板的写法做了“口语化但技术严谨”的重组：保留“技术领域、背景技术、发明内容、附图说明、具体实施方式、说明书摘要”的框架，不含权利要求书，并把我前面指出的三项关键整改一并并入了正文，即资料快照口径统一、保护重点从模型转向流程、以及把时间对齐和缺失证据处理写成明确模块。模板依据来自你上传的专利模板文档。turn0file0

¹ ⁵ ⁸ https://github.com/1144160159/research_detecting

https://github.com/1144160159/research_detecting

- 2 3 13 17 19 21 https://github.com/1144160159/research_detecting/blob/main/%E6%96%B9%E5%90%91%E5%88%86%E6%9E%90/%E4%B8%93%E5%88%A9/%E4%BA%94%E4%B8%AA%E6%96%B9%E5%90%91%E5%88%86%E6%9E%90/%E4%B8%93%E5%88%A9/%E4%BA%94%E4%B8%AA%E6%96%B9%E5%90%91_%E4%B8%93%E5%88%A9
- 4 https://github.com/1144160159/research_detecting/tree/main/%E6%96%B9%E5%90%91%E5%88%86%E6%9E%90/%E4%B8%93%E5%88%A9
- https://github.com/1144160159/research_detecting/tree/main/%E6%96%B9%E5%90%91%E5%88%86%E6%9E%90/%E4%B8%93%E5%88%A9
- 6 https://github.com/1144160159/research_detecting/tree/main/%E7%BB%BC%E5%90%88%E5%88%86%E6%9E%90
- https://github.com/1144160159/research_detecting/tree/main/%E7%BB%BC%E5%90%88%E5%88%86%E6%9E%90
- 7 15 https://github.com/1144160159/research_detecting/blob/main/%E7%BB%BC%E5%90%88%E5%88%86%E6%9E%90/06_%E6%80%BB%E7%BB%93%E6%8A%A5%E5%91%8A.md
- https://github.com/1144160159/research_detecting/blob/main/%E7%BB%BC%E5%90%88%E5%88%86%E6%9E%90/06_%E6%80%BB%E7%BB%93%E6%8A%A5%E5%91%8A.md
- 9 14 <https://arxiv.org/abs/1910.00056>
- <https://arxiv.org/abs/1910.00056>
- 10 22 <https://arxiv.org/abs/2001.01525>
- <https://arxiv.org/abs/2001.01525>
- 11 18 24 25 <https://arxiv.org/abs/2405.02629>
- <https://arxiv.org/abs/2405.02629>
- 12 20 <https://arxiv.org/abs/2101.10474>
- <https://arxiv.org/abs/2101.10474>
- 16 https://www.cnipa.gov.cn/art/2025/11/13/art_99_202568.html
- https://www.cnipa.gov.cn/art/2025/11/13/art_99_202568.html
- 23 <https://arxiv.org/abs/2103.03080>
- <https://arxiv.org/abs/2103.03080>